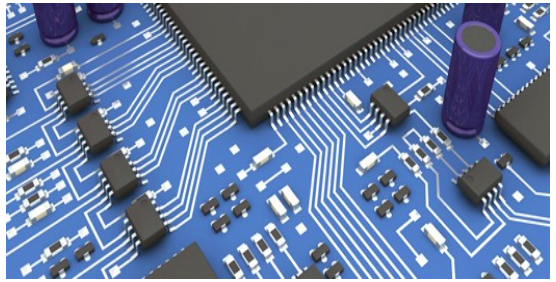
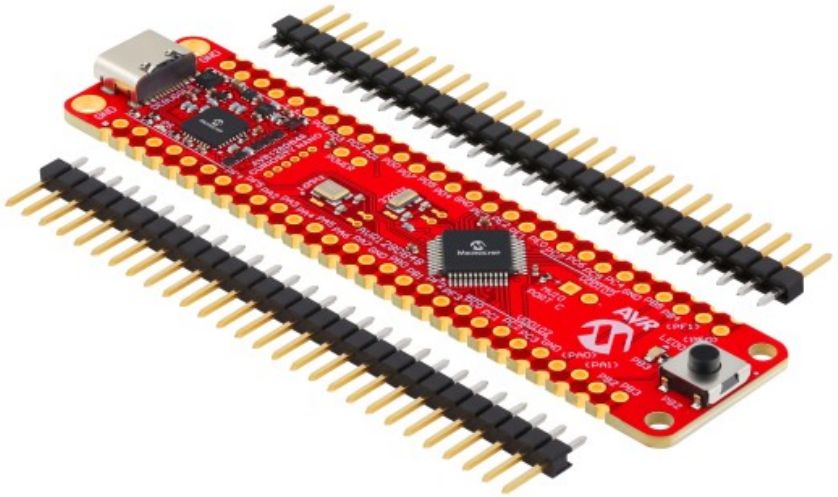
Aplikace Embedded systémů v Mechatronice





MPLAB® X IDE

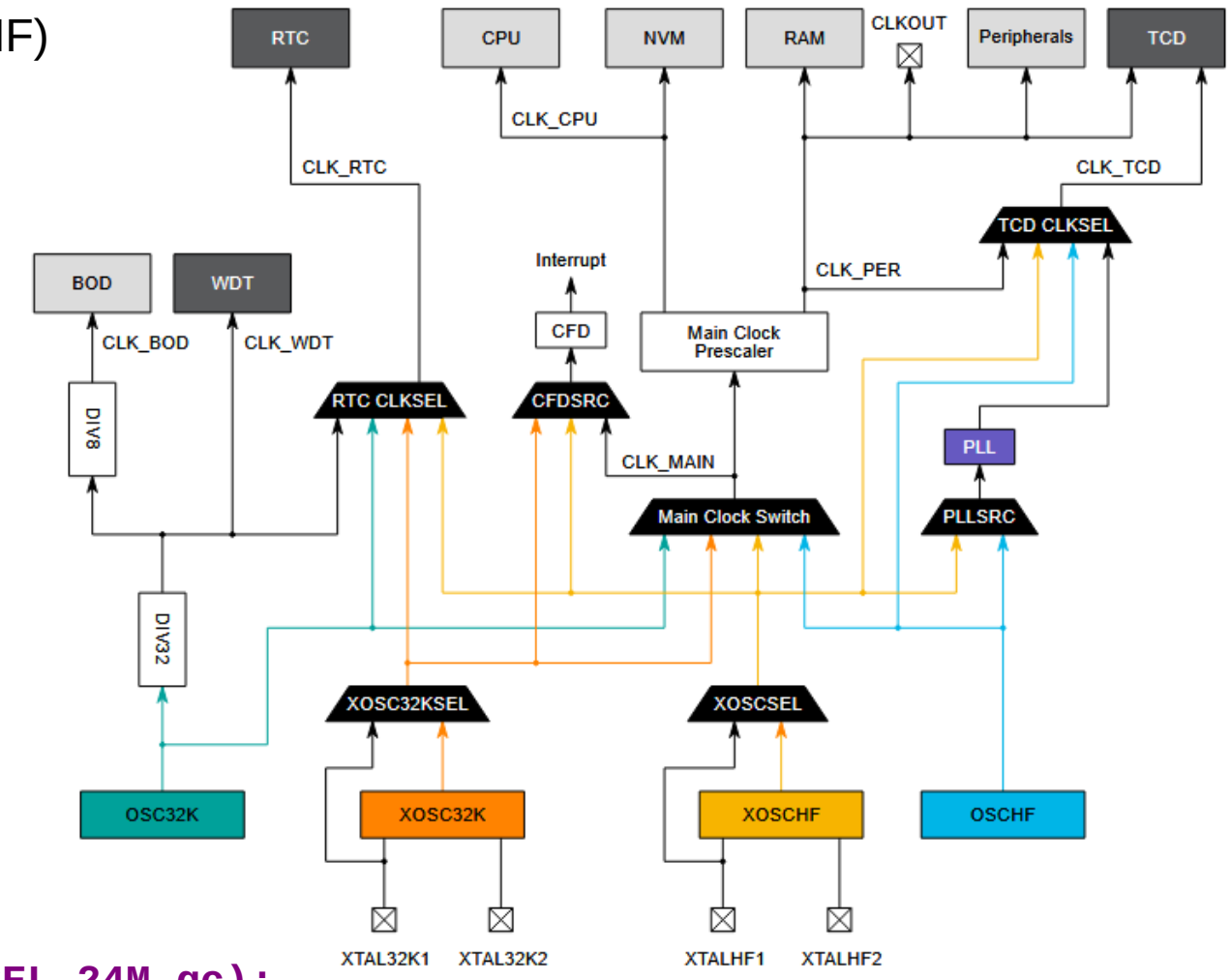
Michal Bastl

Nastavení hlavního oscilátoru

Internal Oscillators:
- Up to 24 MHz Internal High-Frequency Oscillator (OSCHF)
- 32.768 kHz Ultra-Low Power Oscillator (OSC32K)

External Clock Options:
- 32.768 kHz Crystal Oscillator (XOSC32K)
- High-Frequency Crystal Oscillator (XOSCHF)
- External clock

Registry jsou chráněny:
- je třeba využít knihovní funkci pro zápis



Nastavení jako zdroje krystal na pinech:

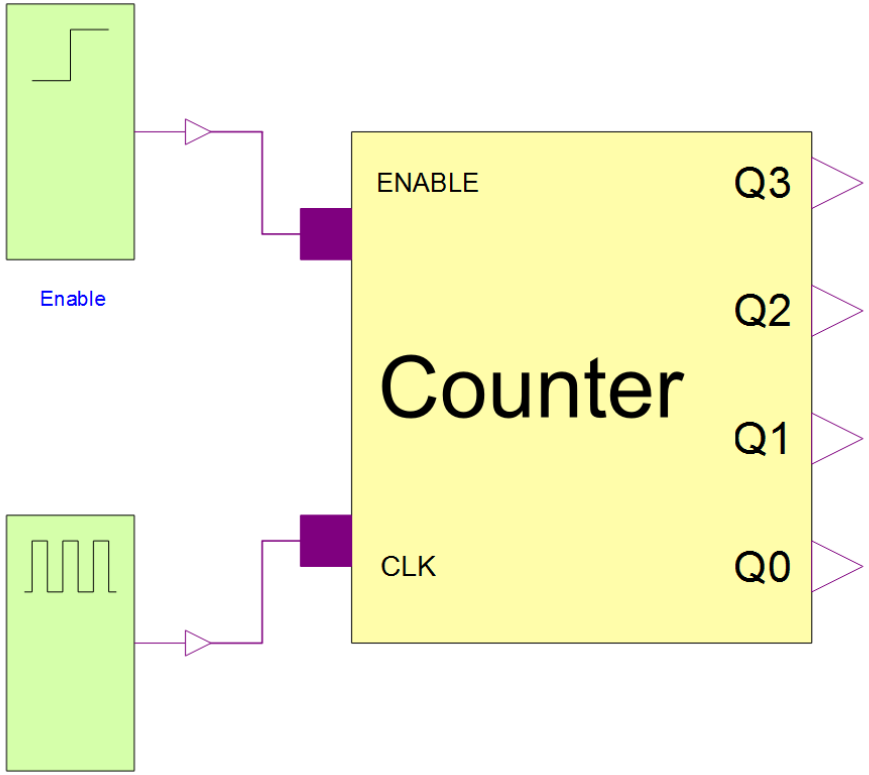
```
_PROTECTED_WRITE(CLKCTRL_XOSCHFCTRLA, 0x05);
_PROTECTED_WRITE(CLKCTRL_MCLKCTRLA, 0x03);
```

Nastavení interního oscilátoru na 24MHz

```
_PROTECTED_WRITE(CLKCTRL_OSCHFCTRLA, CLKCTRL_FRQSEL_24M_gc);
```

Timer

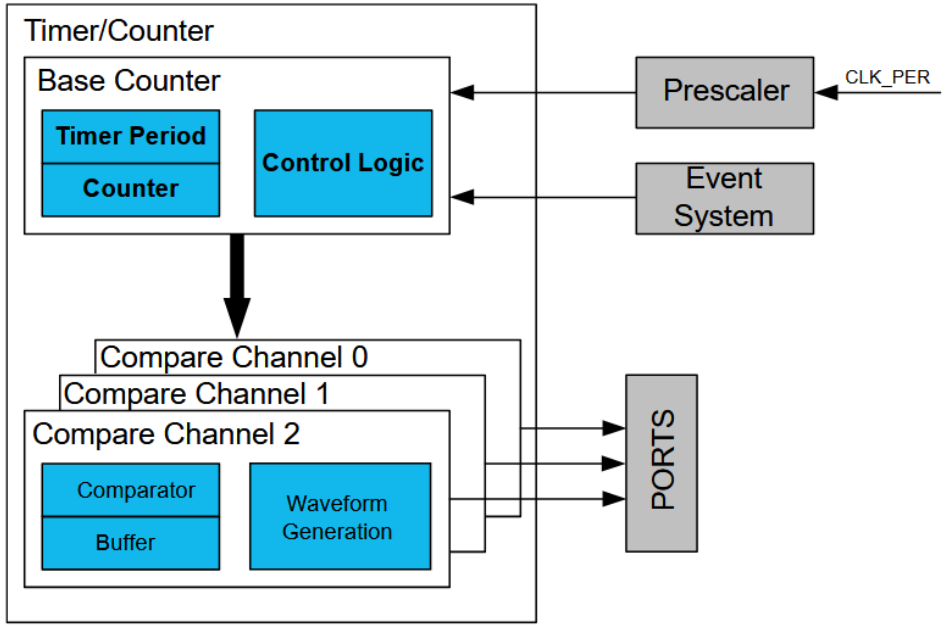
- Timer je periferie MCU. Jedná se o obvod k odměřování času. Jde v podstatě o **binární čítač**, který načítá počet pulzů referenčního signálu.
- Zdroj signálu může být upraven děličkou.
- Aktuální stav čítače je možné vyčíst z příslušných registrů.
- AVR: TCA, TCB, RTC.
- čas se odvozuje od frekvence vstupního signálu.



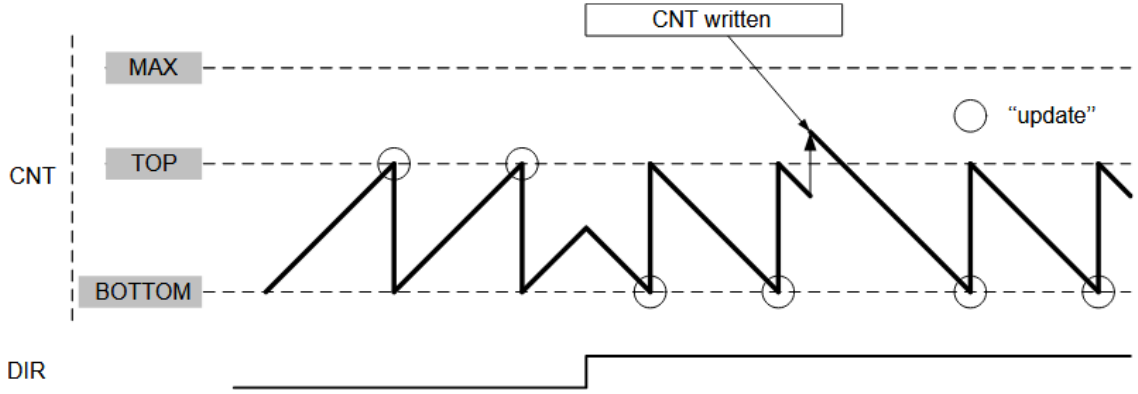
Timer typu TCA

AVR128DB48 obsahu 2xtimer TCA

- 16 – bit timer/counter
- Timer Overflow
- Compare Match
- generování (PWM)

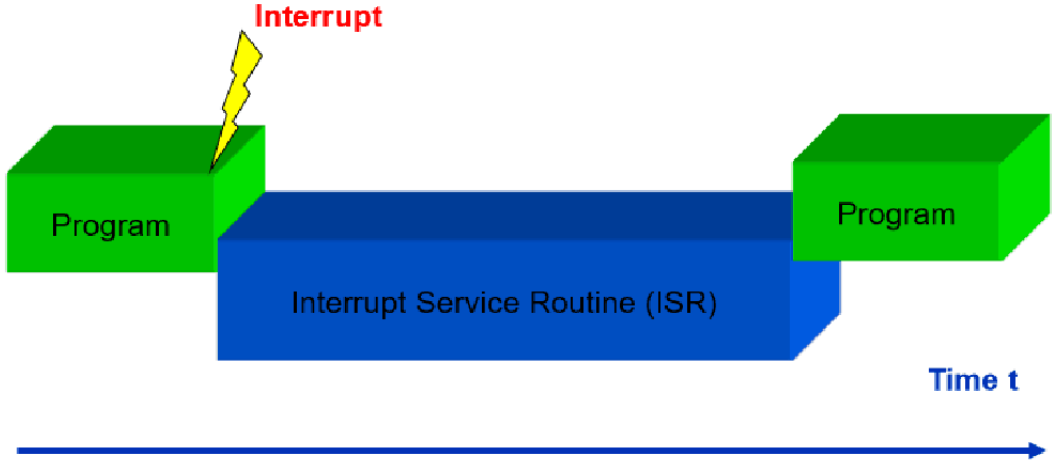


$$PER = \frac{f_{clk}}{f_{des} \cdot Prescaler} - 1$$

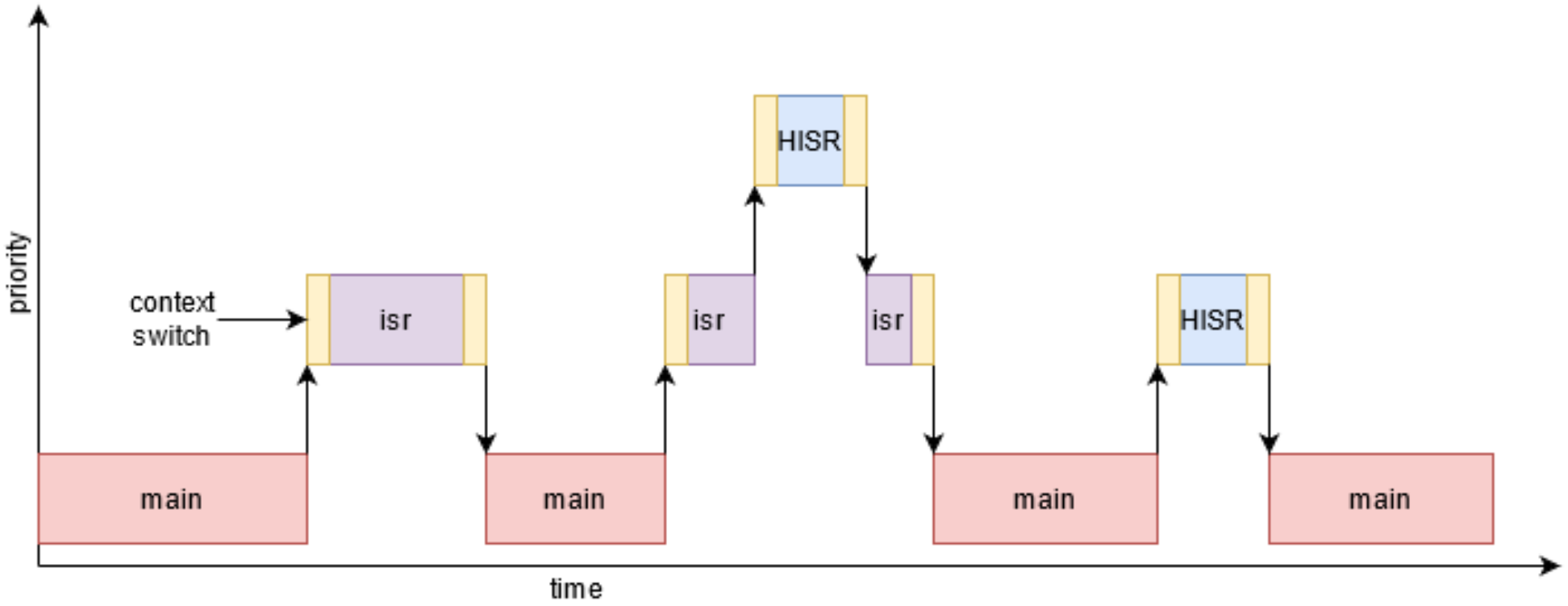


Interrupt

- Je to signál pro CPU, že nastala událost vyžadující okamžitou pozornost.
- MCU obsahuje řadič přerušení, který po vyvolání události přeruší program a obslouží *Interrupt Service Routine*.
- Poslední instrukce je dokončena. Adresa následující je uložena a po návratu z přerušení se pokračuje.
- Přerušení mohou mít priority a mohou být i vnořená.



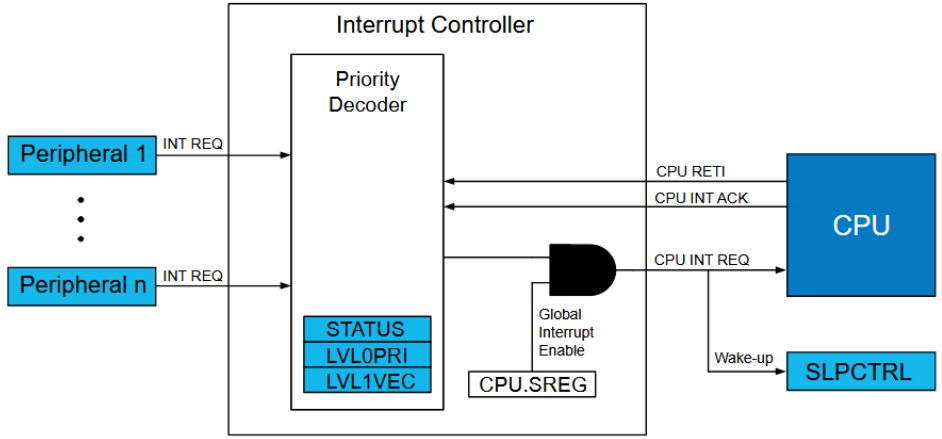
Interrupt


priority
context
switch
isr
HISR
isr
isr
HISR
main
main
main
main
time

Interrupt

- Zdrojem přerušení můžou být různé události.
- Kompletní seznam nalezne uživatel v datasheetu
- Lze zmínit např. přerušení od timeru, ADC převodníků, změna stavu pinu, komunikační sběrnice (příjem dat) apod.
- Ve cvičeních budeme pracovat s přerušením, které je vyvolané periferii TIMER.
- Budeme tedy v přesných časových intervalech vykonávat program, který vložíme do tzv. obsluhy přerušení.
- Tato obsluha se nazývá ISR tedy interrupt-service-rutine.

Figure 15-1. CPUINT Block Diagram



Vector table:

Interrupt Vector Table:

- Jedná se o vyhrazenou oblast v programové paměti (Flash), u AVR od adresy 0x0000. Každý typ přerušení má v této tabulce pevně přidělenou adresu (slot).
- Na těchto adresách se nachází instrukce skoku (JMP) na příslušnou obslužnou rutinu (funkci).
- U AVR pořadí v tabulce určuje prioritu; čím nižší adresa, tím vyšší priorita.

Table 10-3. Interrupt Vector Mapping

Vector number	Program Address (word)	Peripheral Source (name)	Description	28-Pin	32-Pin	48-Pin	64-Pin
0	0x00	RESET		X	X	X	X
1	0x02	NMI	Non-Maskable Interrupt available for: • CRCSCAN • CFD	X	X	X	X
2	0x04	BOD_VLM	Voltage Level Monitor Interrupt	X	X	X	X
3	0x06	CLKCTRL_CFD	External crystal oscillator/clock source failure Interrupt	X	X	X	X
4	0x08	MVIO_MVIO	Multi-Voltage Input/Output Interrupt	X	X	X	X
5	0x0A	RTC_CNT	Real-Time Counter Overflow or Compare Match Interrupt	X	X	X	X
6	0x0C	RTC_PIT	Real-Time Counter Periodic Interrupt	X	X	X	X
7	0x0E	CCL_CCL	Configurable Custom Logic Interrupt	X	X	X	X
8	0x10	PORTA_PORT	PORT A External Interrupt	X	X	X	X
9	0x12	TCA0_OVF TCA0_LUNF	Normal: Timer/Counter Type A Overflow Interrupt Split: Timer/Counter Type A Low Underflow Interrupt	X	X	X	X
10	0x14	TCA0_HUNF	Normal: Unused Split: Timer/Counter Type A High Underflow Interrupt	X	X	X	X
11	0x16	TCA0_CMP0 TCA0_LCMP0	Normal: Timer/Counter Type A Compare 0 Interrupt Split: Timer/Counter Type A Low Compare 0 Interrupt	X	X	X	X

Zápis ISR

- Zápis se provádí vytvořením speciální funkce tzv. *Interrupt Service Routine.*
- U AVR pomocí knihovny <avr/interrupt.h>
- Mohu používat lokální i statické proměnné.
- Globální proměnné, které používám jak v ISR, tak zbytku programu musí být volatile.
- Tuto funkci ISR nelze volat z kódu. Provádí se s příznakem přerušení Interrupt flag!
- Množství kódu v ISR se snažím omezovat.
- Pokud nedojde ke smazaní příznaku (flag), ISR se vyvolá okamžitě znovu!

```
ISR(TCA0_OVF_vect){

    TCA0.SINGLE.INTFLAGS = TCA_SINGLE_OVF_bm;

    PORTB.OUTTGL = PIN3_bm;

}
```

Existuje globální bit pro povolení přerušení v registru SREG.
Globální povolení přerušení můžeme udělat makrem sei()
Zákaz je možný pomocí cli()

Pokud používám v ISR i hlavním programu:

```
ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
        millis_now = millis;
    }
```

Interrupt - proměnné

- Předání informací mezi hlavním programem a obsluhou přerušení se provádí pomocí globálních proměnných.
- Tyto proměnné musí být typu volatile, což zakazuje proměnnou optimalizovat.
- Změna této proměnné je vyvolána asynchronní událostí.
- Fakt, že se proměnná mění z více míst programu je častý zdroj problémů a chyb.

```
volatile uint32_t millis = 0;

ISR(TCA0_OVF_vect){

    TCA0.SINGLE.INTFLAGS = TCA_SINGLE_OVF_bm;

    millis++;
}
```

```
// nebude preruseno
ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
    millis_now = millis;
}
```

Použití TCA

```
PORTB.DIRSET = PIN3_bm;

// zapnout preruseni
TCA0.SINGLE.INTCTRL = TCA_SINGLE_OVF_bm;

// nastavit period registr
TCA0.SINGLE.PER = 999;

// nastaveni delicky a zapnuti timeru
TCA0.SINGLE.CTRLA = TCA_SINGLE_CLKSEL_DIV4_gc | TCA_SINGLE_ENABLE_bm
```

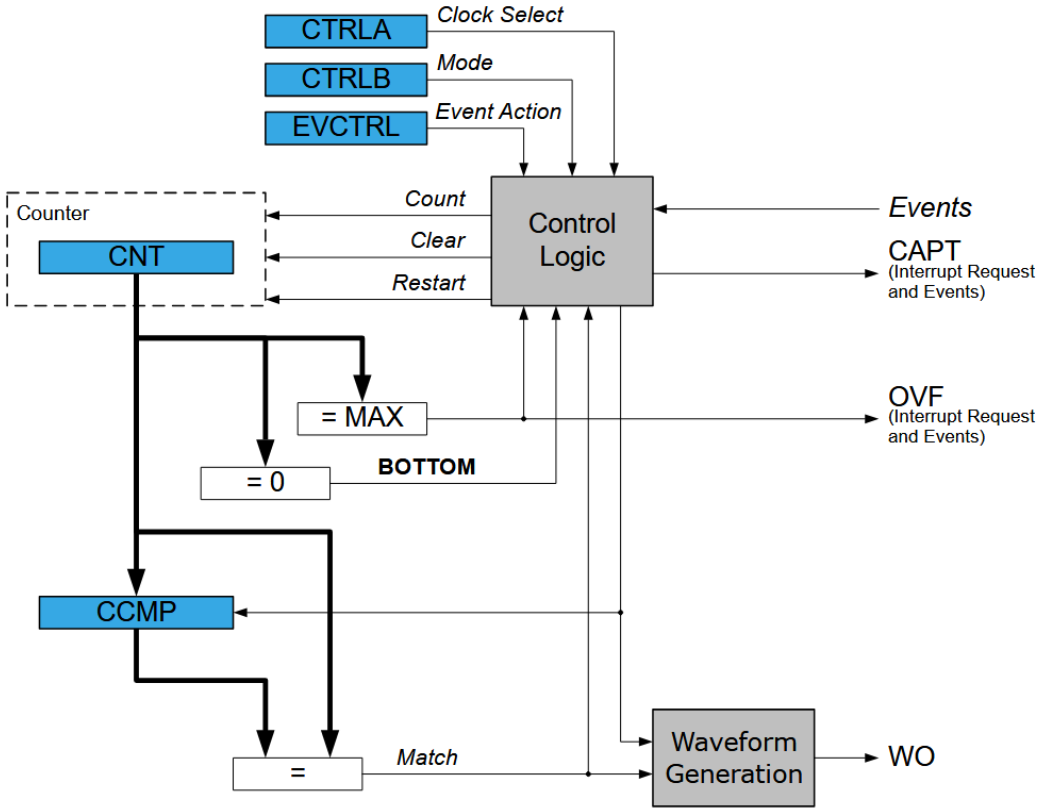
```
uint32_t millis_prev = 0;
uint32_t millis_now = 0;

sei();

while(1){
    // nebude preruseno
    ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
        millis_now = millis;
    }
```

```
volatile uint32_t millis = 0;

ISR(TCA0_OVF_vect){

    TCA0.SINGLE.INTFLAGS = TCA_SINGLE_OVF_bm;

    millis++;
}
```

Timer TCB

AVR128DB48 obsahu 4xtimer TCB

- 16 – bit timer/counter
- Timer Overflow
- Compare Match
- Input capture (měření, frekvence, PWM)



$$CCMP = \frac{f_{clk}}{f_{des} \cdot Prescaler} - 1$$

Použití TCB

```
// PB3 jako out
PORTB.DIRSET = PIN3_bm;

// nastaven cmp na 1kHz
TCB0.CCMP = 3999;

// mod timeru je periodic interrupt
TCB0.CTRLB = TCB_CNTMODE_INT_gc;

// povleni interruptu na CMP
TCB0.INTCTRL = TCB_CAPT_bm;

// zapnuti a nastaveni div
TCB0.CTRLA = TCB_CLKSEL_DIV1_gc | TCB_ENABLE_bm;

// globalni zapnuti
sei();
```

```
ISR(TCB0_INT_vect) {
    // smazat flag
    TCB0.INTFLAGS = TCB_CAPT_bm;

    PORTB.OUTTGL = PIN3_bm;
}
```

Externí přerušení: změna na pinu

Lze vyvolat přerušení na změnu GPIO:

- Vzestupná/sestupná hrana (nebo obojí)
- Jen jeden vektor pro celý PORT
- V ISR lze kontrolovat příslušný pin
- Je třeba smazat flag!

ISC[2:0] Input/Sense Configuration

This bit field controls the input and sense configuration of pin n. The sense configuration determines how to trigger a port interrupt.

Value	Name	Description
0x0	INTDISABLE	Interrupt disabled but digital input buffer enabled
0x1	BOTHEDGES	Interrupt enabled with sense on both edges
0x2	RISING	Interrupt enabled with sense on rising edge
0x3	FALLING	Interrupt enabled with sense on falling edge
0x4	INPUT_DISABLE	Interrupt and digital input buffer disabled(1)
0x5	LEVEL	Interrupt enabled with sense on low level(2)
other	—	Reserved

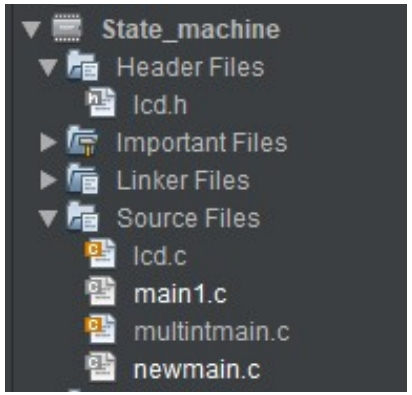
```
void io_init(void) {
    // PB3 out
    PORTB.DIRSET = PIN3_bm;

    // PB2 je vstup
    PORTB.DIRCLR = PIN2_bm;

    // PULL-UP a preruseni na sestupnou hranu
    PORTB.PIN2CTRL = PORT_PULLUPEN_bm | PORT_ISC_FALLING_gc;

}
```

```
ISR(PORTB_PORT_vect) {
    // Kontrola, zda přerušení způsobil PIN2 (pomocí registru INTFLAGS)
    if (PORTB.INTFLAGS & PIN2_bm) {

        // Otočení (toggle) stavu pinu PB3
        PORTB.OUTTGL = PIN3_bm;

        // Vynulování příznaku přerušení pro PIN2
        PORTB.INTFLAGS = PIN2_bm;
    }
}
```

LCD

Parametry:

Řádky x znaky:	2x16
Standard:	HD44780
Převodník i2c:	PCF8574

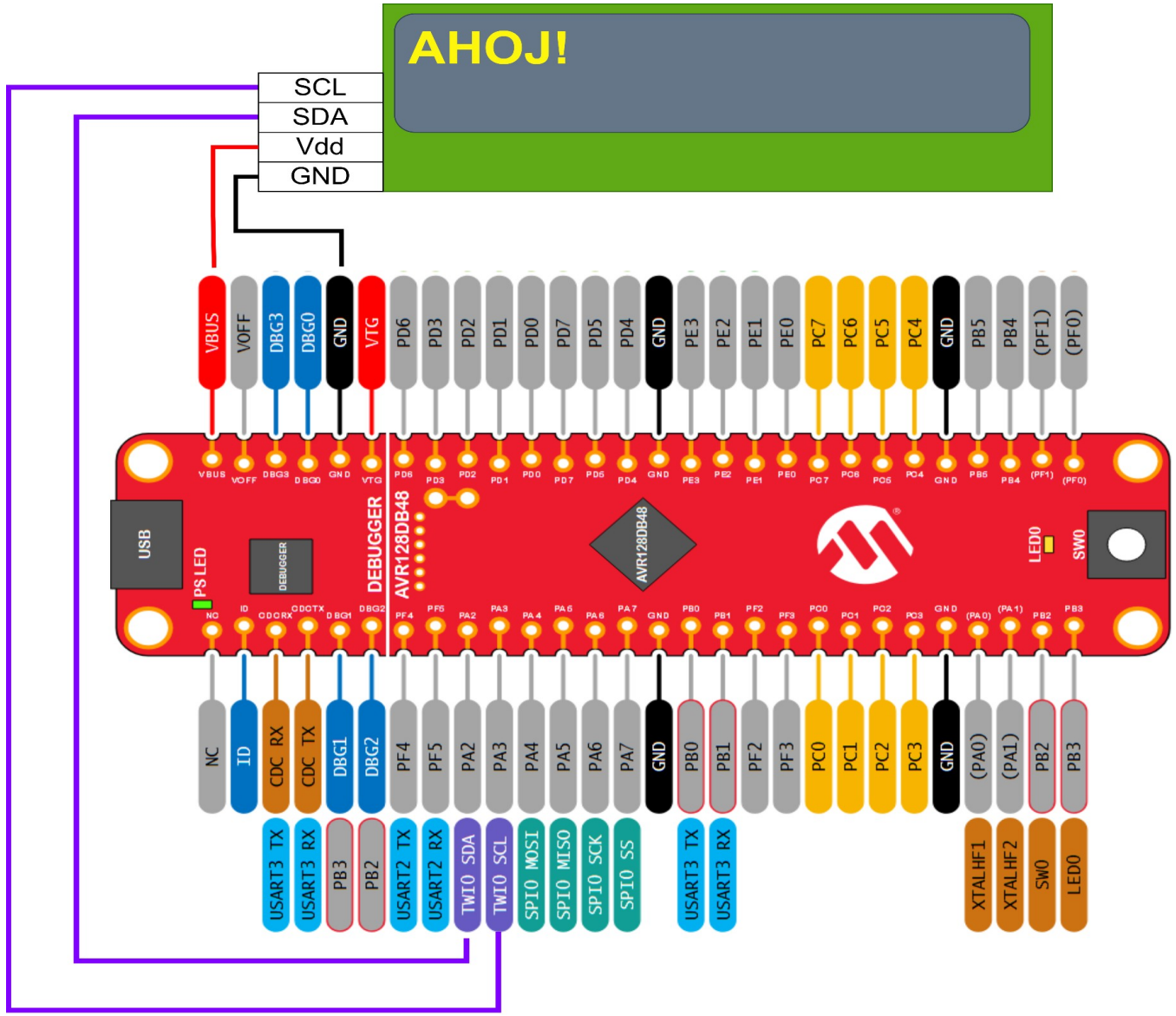


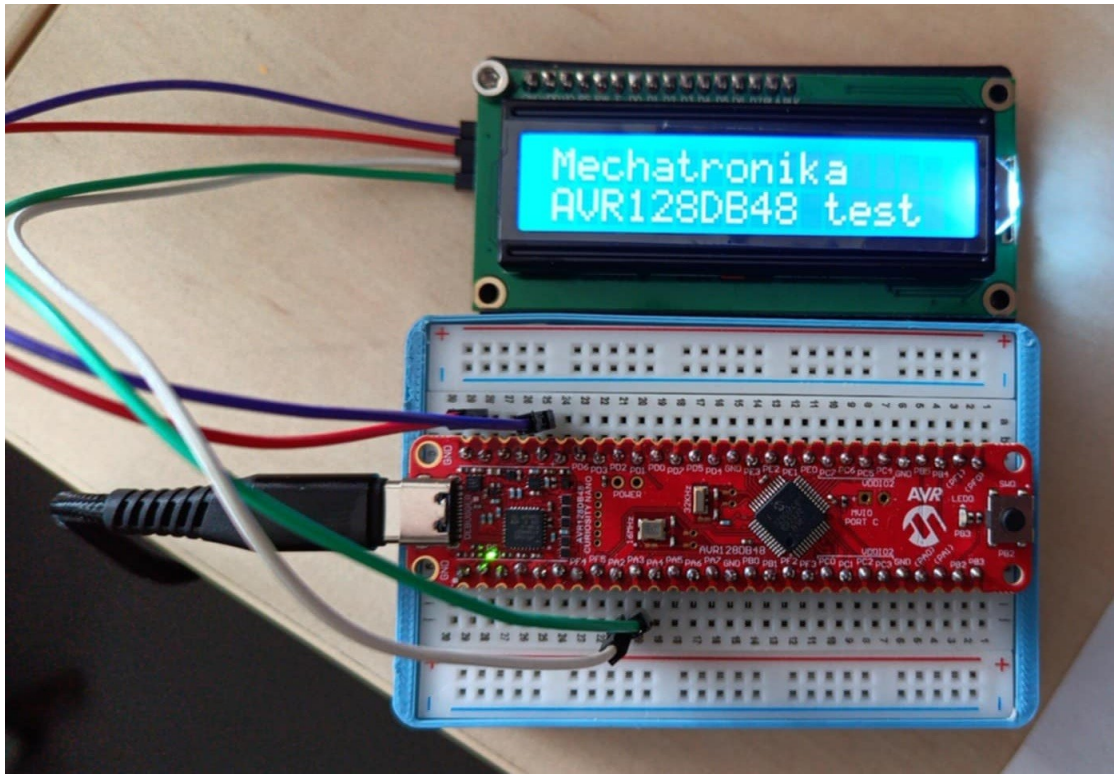
Jak zahrnout do projektu:

1) Rozbalit vložit lcd.h a lcd.c do složky projektu
2) Vložit do header a source files (existing item)
3) include lcd.h

Ukázka zahrnutí knihovny do projektu

LCD


AHOJ!
SCL
SDA
Vdd
GND
VBUS
VOFF
DBG3
DBG0
GND
VTG
PD6
PD3
PD2
PD1
PD0
PD7
PD5
PD4
GND
PE3
PE2
PE1
PE0
PC7
PC6
PC5
PC4
GND
PB5
PB4
(PF1)
(PF0)
USB
PS LED
DEBUGGER
AVR128DB48
LED0
SW0
NC
ID
CDC RX
CDC TX
DBG1
DBG2
PF4
PF5
PA2
PA3
PA4
PA5
PA6
PA7
GND
PB0
PB1
PF2
PF3
PC0
PC1
PC2
PC3
GND
(PA0)
(PA1)
PB2
PB3
USART3 TX
USART3 RX
PB3
PB2
USART2 TX
USART2 RX
TWI0 SDA
TWI0 SCL
SPI0 MOSI
SPI0 MISO
SPI0 SCK
SPI0 SS
USART3 TX
USART3 RX
XTALHF1
XTALHF2
SW0
LED0


Mechatronika
AVR128DB48 test